

2.1 The Vision

The basic idea of EcoGrid Energy is a peer-to-peer trading platform for renewable energy produced by households equipped with solar panels. The platform shall facilitate trading of excess energy with nearby users and also enable collection of smart-meter data and corresponding financial settlement.

We want the business areas to develop freely without tight coupling between them. Marketplace functions, smart-meter processing and financial settlement of trades are very different and therefore require different approaches and have different loads [1], [2].

So, the above-mentioned areas can grow separately and also without too much unnecessary complexity the whole system needs to be of practical use for the founding engineering team of four people. And of course, also evolve further as required.

2.2 Architectural Drivers

High-Frequency Smart-Meter Data

Large volumes of smart meter data will be generated by households connected to EcoGrid. That data needs to be processed with sufficient throughput and scalability of the meter data processing [2]. Any processing delays could also be problematic since other parts of the EcoGrid system may depend on timely readings from the smart meters [2].

Separation of Different Workloads

The workloads of Marketplace activities and the smart-meter ingestion service are very different. Market activities are based mostly on user actions, such as creating an offer or a trade. Smart-meter ingestion on the other hand deals with a continuous flow of events (meter-readings), possibly in high volumes [2]. Thus, a large increase in meter readings should not significantly degrade the performance or availability of the Marketplace [2].

Financial Integrity and Security

Financial Settlement: This function requires precise and reliable processing of all transactions and therefore must have mechanisms to handle transaction consistency, error recovery, duplicate detection and idempotent processing, and data protection [1], [3].

Maintainability and Evolvability

As a new platform, EcoGrid will have evolving requirements for its business. Clear system boundaries and low coupling will allow one area of the platform to evolve independently of the others [1]. Also important is the ability to monitor the system and write automated tests in order to catch failures and test for architectural requirements as they evolve [1].

AI Assisted design portfolio

2.1 The Vision

Stage	Prompt	AI Response Summary	Accepted	Modified	Rejected	Reason
Initial Attempt	What should the overall software architecture vision focus on for a peer-to-peer renewable energy trading platform?	Main ideas included keeping the vision high level, identifying the major business areas, and making sure the platform can remain manageable as it grows. Detailed technology choices were treated as something to decide later.	No	Yes	No	Useful as a starting point, but it covered the platform at a fairly general level. I still needed to understand how the different parts of <u>EcoGrid</u> would influence the architecture vision.
Refinement 1	The platform has a few quite different areas, like the marketplace, smart-meter processing and financial settlement. Does that change what I should be thinking about	Different responsibilities and workloads were identified as an important reason for having clear boundaries between the main areas. <u>Loose</u>	No	Yes	No	This gave me a clearer direction. However, I wanted to look more closely at why loose coupling would actually help when

Stage	Prompt	AI Response Summary	Accepted	Modified	Rejected	Reason
	for the architecture vision? Keep the answer brief. Give me 2–4 short ideas only. I'm trying to understand the concept, not generate report paragraphs.	coupling was also raised so that changes in one area would have less impact on the others.				the areas need to grow or change at different rates.
Refinement 2	If those areas have different responsibilities and different workloads, why might it be useful to keep them loosely coupled so that one area can change or grow without strongly affecting the others?	Key benefits mentioned were independent scaling, safer changes, better fault isolation, and allowing each area to evolve at its own pace. Smart-meter processing was used as an example of an area that may need more capacity than the marketplace.	No	Yes	No	At this point the benefit of separating the areas was much clearer. The remaining question for me was how far to take that separation without making the design harder for a small team to manage.
Refinement 3	If we want those areas to	The main suggestion	Yes	No	No	This covered the

Stage	Prompt	AI Response Summary	Accepted	Modified	Rejected	Reason
	grow independently, how should that be balanced with keeping the architecture reasonably simple for a small engineering team, while still allowing the system to evolve later?	was to keep clear boundaries without splitting the platform into too many small services too early. Separation should have a clear reason, while the initial design should stay manageable and still leave room for later growth.				balance I was trying to understand. It connected independent growth with simplicity, team size, and future evolution, so I had enough information to form my own architecture vision.

2.2 Architectural Drivers

Stage	Prompt	AI Response Summary	Accepted	Modified	Rejected	Reason
Initial Attempt	What are some things that could really affect the architecture of a renewable energy trading	Four broad concerns came up: frequent smart-meter data, different workload types, security and	No	Yes	No	A useful overview, but it was still quite general. I wanted to look more closely at why continuous meter readings could become

Stage	Prompt	AI Response Summary	Accepted	Modified	Rejected	Reason
	platform that deals with smart-meter data and trading? Keep it brief.	financial transactions , and the need for the platform to scale and evolve.				an architectural issue.
Refinement 1	One thing I'm thinking about is that lots of households could be sending meter readings all the time. What problems could that cause for processing speed, scaling and keeping the readings up to date? give me a little bit more detail here but not too big. explain to me but dont be hollow	Continuous readings could create backlogs if processing cannot keep up. More households may require extra processing capacity, and delays could leave other parts of the system using older readings. Traffic spikes were also mentioned as another possible issue.	No	Yes	No	This gave me enough detail to understand the high-frequency data problem. The next thing I needed to understand was whether that heavy meter workload could interfere with the marketplace side of the system.

Stage	Prompt	AI Response Summary	Accepted	Modified	Rejected	Reason
Refinement 2	The meter side seems pretty continuous, but marketplace activity mostly happens when users actually do something. If those workloads are really different, why could it be a problem if they affect each other too much?	Heavy meter traffic could consume shared CPU, database connections or network capacity and slow down marketplace actions. Tight dependency could also make failures spread between the two areas, force them to scale together, and make changes harder to manage.	No	Yes	No	The difference between the two workloads became much clearer here. It also showed why some isolation is useful. I still had to consider concerns beyond performance, especially because the platform also processes financial transactions and will change over time.
Refinement 3	Apart from those workload and performance issues, the platform also has financial settlement	Other important areas included handling failed transaction steps safely, protecting settlement	Yes	No	No	This filled in the areas that were still missing from my understanding . I now had the main concerns around performance,

Stage	Prompt	AI Response Summary	Accepted	Modified	Rejected	Reason
	and will probably change as it grows. What other architecture concerns should I be thinking about around reliable transactions, security and keeping the system easy to change?	data, keeping audit records, using clear boundaries and low coupling for easier change, and having enough observability to notice failures or unusual behaviour.				workload separation, reliable financial processing, security, and keeping the platform maintainable as it grows.

References

- [1] “Software Architecture in Practice, 4th Edition | CMU Software Engineering Institute.” Accessed: Sep. 24, 2026. [Online]. Available: <https://www.sei.cmu.edu/library/software-architecture-in-practice-fourth-edition/>
- [2] claytonsiemens77, “Event-Driven Architecture Style - Azure Architecture Center.” Accessed: Sep. 24, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven>
- [3] claytonsiemens77, “Saga Design Pattern - Azure Architecture Center.” Accessed: Sep. 24, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/saga>